Assignment 1: Conceptual Architecture of OpenCV

October 10, 2025

Website :
https://softwarearchitecture.wixsite.com/eecs4314assignment
Group Name: SourceArchitects
Team Members Information

Name	Student ID	Email
Sheida Khodabakhshian Khansari	215534340	sheidakh@my.yorku.ca
Vrinda Ashwar	218624692	vrinda03@my.yorku.ca
Matthew Lee	218256743	mattl7@my.yorku.ca
Werner Bagu-Paiz	21727772	wernerbp@my.yorku.ca
William Moran	218928960	willvm24@my.yorku.ca
Stefewn Johnson	219411099	stef9@my.yorku.ca

Abstract 2
Introduction & Overview: 2
Architecture: 2
- **System Components & Interaction:** 3
- **Conceptual Architecture:** 6
- **Evolution of the System:** 7
- **Control & Data Flow between System Components:** 8
  - Data Flow 9
  - Control Flow 9
- **Concurrency in the System:** 9

Division of Responsibilities Among Developers: 10
Sequence Diagrams Based on Conceptual Architecture: 12
Conclusions: 13
Lessons Learned: 13
References: 14

Abstract

The following report provides an overview of the conceptual architecture of OpenCV, one of the most widely recognized open-source libraries in computer vision. It describes and defines the interactions between the system's major components. These components include the modules responsible for Core functionality, Image I/O, Video I/O, Image Processing, High-level GUI, Video Analysis, Image Stitching, Camera Calibration and 3D Reconstruction, 2D Features Framework, Object Detection, Deep Neural Network module, Machine Learning, Clustering and Search in Multi-Dimensional Spaces (flann), Computational Photography, and Graph API. This modular structure of OpenCV was examined and the conceptual architecture most aligned with this system was found to be the layered architecture style.Although, it does not follow a strict top-to-bottom dependency.However, the architecture style used within each module varies. The module responsible for image stitching uses a pipe-and-filter architecture style while the Graph API module uses the layered architecture. System evolution, data flow based on a pipeline architecture, and concurrency are explored in this paper as well. OpenCV uses concurrency through built-in support for parallelization, such as using cv::parallel_for, and backend- frameworks such as TBB and OpenMP, OpenCV [1]. An example of image processing is explored through image processing tasks across multiple threads [2]. This paper also provides an overview of the division of responsibilities among the system's key stakeholders including the communicator, developers, testers, users and integrators. It also illustrates the use cases of OpenCV in OpenDataCam and robotics through sequence diagrams for real-time object detection and counting in OpenDataCam and movement guidance in robotics.

Introduction & Overview:

OpenCV or Open Source Computer Vision Library is a software library first developed by Intel in 1999 and released in 2000. It is an Apache 2 licensed open source library and since 2012 it has been maintained by a community of developers under the OpenCV.org, a non-profit organization[3],[4]. This software library contains over 2500 algorithms designed for a range of computer vision and machine learning applications [3]. It is capable of tasks such as face and object recognition, action classification in videos, motion tracking, 3D model and point cloud generation, image stitching, image similarity search, eye-tracking, scene recognition, augmented reality integration and many more computer vision applications[3]. This library has become one of the most recognized and utilized tools in computer vision [4] It offers interfaces for C++, Python, Java, and MATLAB, and is compatible with Windows, Linux, Android, and macOS platforms[3]. The following report will provide an overview of the conceptual architecture of OpenCV. It will delve into its major components and their interactions including control and data flow among the parts as well as concurrency. It will also provide information on the evolution of OpenCV and the division of responsibilities among the key stakeholders. Sequence diagrams will also be provided to give an overview of noteworthy use cases in OpenDataCam and robotics.

Architecture:

As mentioned, OpenCV is a computer vision library. Computer vision is a branch of artificial intelligence that enables machines to analyze and interpret visual data[5]. The computer vision pipeline outlines the steps for processing and analysis of visual data. These main steps include image acquisition, preprocessing, image segmentation,

feature extraction,object detection,object recognition and classification,post processing, as well as visualization and interpretation [5]. Based on these steps of the computer vision pipeline as well as through review of existing OpenCV documentation , the high-level conceptual architecture of OpenCV can be reconstructed.

Based on the provided documentation, OpenCV has a modular structure. There are numerous modules and each is dedicated to specific functionalities within computer vision and machine learning. The modular structure of OpenCV promotes extensibility, allowing developers to easily incorporate custom modules or third-party contributions. This flexibility enables the library to evolve continuously by supporting new features, tools, and integrations without requiring changes to its core structure [3], [4],[6],[7].

System Components & Interaction:

There are fifteen main modules in OpenCV. The following is the description of each module, their major functions, and their interaction with other modules.

The "**core**" module provides the core functionality of OpenCV. It offers functions and algorithms that other modules depend on including fundamental data structures, matrix operations, and utility functions. This module is critical to several of OpenCV's functionalities related to handling image data, performing mathematical computations. It also serves a role in managing memory efficiently in OpenCV [6],[8]. Thus, any module that performs operations on image data or other fundamental OpenCV types inherently depends on this core module.

The "**imgcodecs**" module has functions for reading and writing to image files[5]. It depends on the "core" module for fundamental structures like cv::Mat that imgcodecs uses. Mat represents an array that can be used to store images[9] . The "**imgproc**" module handles image processing. It provides an extensive set of image processing functions enabling tasks such as image filtering, geometric image transformation, drawing functions, color space conversions, planar subdivision, histogram operations, structural analysis and shape descriptors, motion analysis, and many more [10]. Since the core module stores the fundamental data structures that represent image data, image processing only depends on this module and it doesn't depend on the module that handles image file reading and writing.

The "**videoio**" module handles reading and writing video or image sequences (video I/O) [6],[11]. As shown in figure 1 below, this module offers the cv::VideoCapture and cv::VideoWriter classes as a two-layer interface to various video I/O APIs used as backends [12].

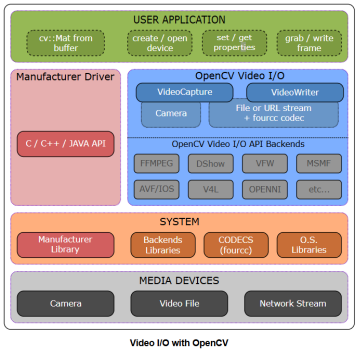

Figure 1.Video I/O with OpenCV adapted from "OpenCV: Video I/O with OpenCV Overview" [12].

The “**video**” module has functions that handle video analysis such as motion estimation, background subtraction, and object tracking algorithms . It is dependent on the image processing module “imgproc” because a video is a sequence of images (frames) and it can use the functionalities provided in the image processing module to analyze each video frame and integrate the results leading to less code redundancy [6].

The “**calib3d**” module provides algorithms that can handle camera calibration and 3D reconstruction including multiple-view geometry algorithms, single and stereo camera calibration, object pose estimation, stereo correspondence algorithms, and elements of 3D reconstruction. In order to handle such tasks it is dependent on functions provided in the image processing module as well as the modules “features2d” and “flann”. The image processing module offers basic tools for preparing and improving images, while features2d and flann deliver intermediate tools for identifying accurate pixel matches. Building on these, the calib3d module can then address advanced tasks in geometry and 3D vision [13].

The “**features2d”** module is a 2D features framework and provides functionalities for detecting,describing and matching 2D features within images. It is dependent on the image processing module as image preprocessing tasks such as conversion to grayscale, and smoothing images to reduce noise and is often essential for feature detection [6][14].

The **“flann”** module handles clustering and search in multi-dimensional spaces. It provides an interface to the “Fast Library for Approximate Nearest Neighbors” which contains algorithms designed to optimize nearest neighbour search in large datasets and for high dimensional features . Flann operates on feature descriptors provided by the 2D features framework and thus doesn’t have dependency on the image processing modules[15].

The “**dnn”** is the deep neural network module that allows for the deployment and execution of pre-trained deep learning models within OpenCV. However, it doesn’t support network training [16]. It is dependent on the image processing module for functions that handle image transformation needed before feeding the image to a neural network as raw images can be noisy,inconsistent, and inefficient for the neural network to process [17].

Furthermore, the machine learning module “**ml**” offers a collection of classes and functions designed for statistical classification, regression, and clustering operations [5]. It is dependent on the image processing module as machine learning models need images to be preprocessed[18].

Computational photography is handled by the “**photo**” module which includes photo processing algorithms that can implement functionalities such as image denoising, image inpainting, and high dynamic range image processing. It is dependent on the image processing modules as it builds on the functionalities provided by this module [19].

Image stitching is also handled by a module called “**stitching**” which contains functions for image stitching and panorama creation. The stitching pipeline is shown in the figure below[20]. It is a high level module that has dependency on the modules that handle image processing (for resizing) ,2D features framework and flann(for finding and matching features), as well as camera calibration and 3D reconstruction(for camera parameter estimation) module. The architecture style of this module is pipe and filter as

shown in figure 2 where input images are processed through different stages which can be categorized into registration and compositing[20].

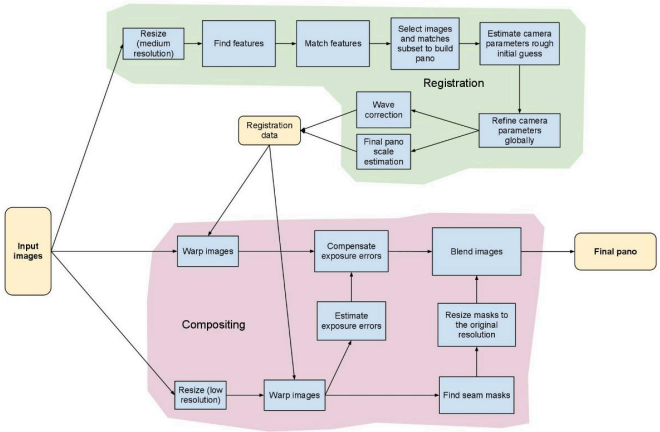

Figure 2. stitching pipeline adapted from "https://docs.opencv.org/4.x/d1/d46/group__stitching.html" [20]

Object detection is handled by the "**objdetect"** module. It provides tools that can identify and locate instances of predefined classes of objects such as faces, eyes, mugs, people, cars,etc.,within image or video streams [6]. It has required dependencies on the image processing module as processing tasks like resizing, grayscale conversions and applying noise reduction techniques are often necessary to to prepare images for analysis by object detection algorithms [21].

The "**highgui"** module handles high-level gui for displaying images and videos and handling user input such as mouse events and keyboard commands. Thus it is dependent on the image processing module, image file reading and writing and video I/O modules to load images and videos, decode and process them [22].

The "**gapi**" module, also known as Graph API, is a high-level module introduced in OpenCV 4.0. It serves as a framework that provides a single API to program image processing pipelines across different backends. Through a graph-based model of execution, it is able to improve the efficiency and portability of regular image processing tasks [23],[24]. Thus it is dependent on the image processing module as it extends this module's capabilities . This module has a layered architecture as shown in figure below. It contains 3 layers; the top layer is the "API layer" which implements the G-API public interface through which users can define and interact with image processing pipelines. This layer contains the building blocks and semantics of G-API like cv::GMat or cv::GComputation . The intermediate layer is the "Graph Compiler" which converts user computations into graphs and applies transformation such as optimizations. The backend layer is the lowest layer where the processed graphs are executed for a specific platform or device[23].

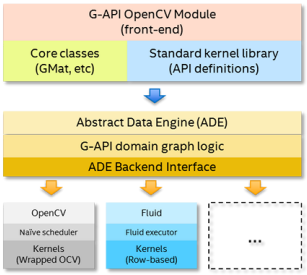

Figure 3.G-API framework architecture adapted from "https://docs.opencv.org/4.x/de/d4d/gapi_hld.html"[23].

Conceptual Architecture:

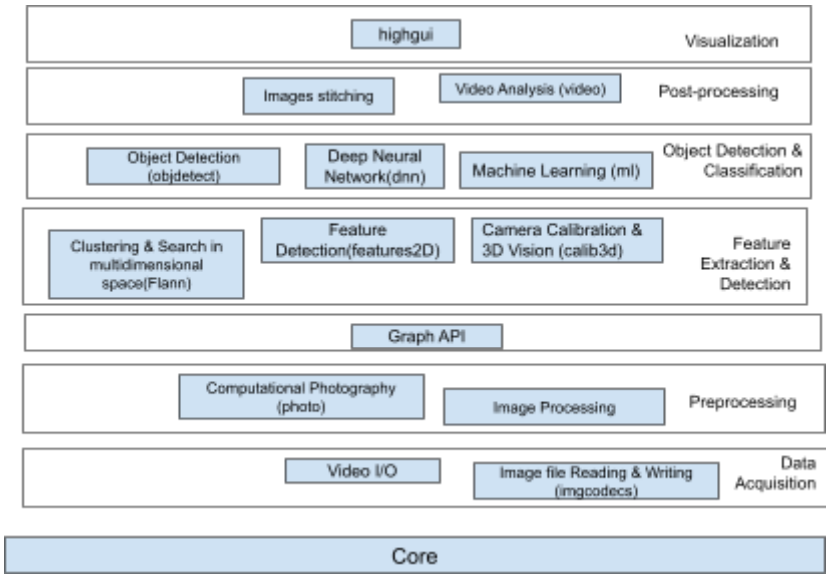

Figure 4: Conceptual Architecture of OpenCV.

OpenCV's modules interact and build upon each other's functionalities to perform complex computer vision tasks. These modules can be organized hierarchically based on their functionality, dependencies, and correspondence to the main stages of the computer vision pipeline. At the foundation is the Core module, which provides essential data structures and mathematical operations, which corresponds to the data representation foundation of the pipeline. The first functional layer, comprising Video I/O and Image File Reading & Writing (imgcodecs) modules, aligns with the data acquisition step, as it handles reading, writing, and capturing visual data from various sources. The second layer includes the Image Processing and Computational Photography (photo) modules, which perform preprocessing operations such as filtering, denoising, and image enhancement to prepare data for higher-level analysis. The third layer contains the Graph API module. The fourth layer, containing modules like Feature Detection (features2D), Camera Calibration & 3D Vision (calib3d), and FLANN for clustering and search, corresponds to the feature extraction and detection stage, where key image features are identified and represented for subsequent analysis. The fifth layer represents the object detection and classification phase, comprising the Object Detection (objdetect), Deep Neural Network (dnn), and Machine Learning (ml) modules

that enable recognition, categorization, and decision-making based on extracted features. The sixth and seventh layers correspond to post-processing and visualization, including modules such as Video Analysis (video), Image Stitching, and HighGUI, which support visualization, user interaction, and integration of processed results into complete applications. Overall, the architecture of OpenCV can be described as modular and layered, where higher-level modules depend on lower-level ones, reflecting the sequential flow of the computer vision pipeline. Some modules, such as the Graph API (G-API), exhibit internal layered or dataflow (pipe-and-filter) architectures, further reinforcing OpenCV's flexibility and scalability.

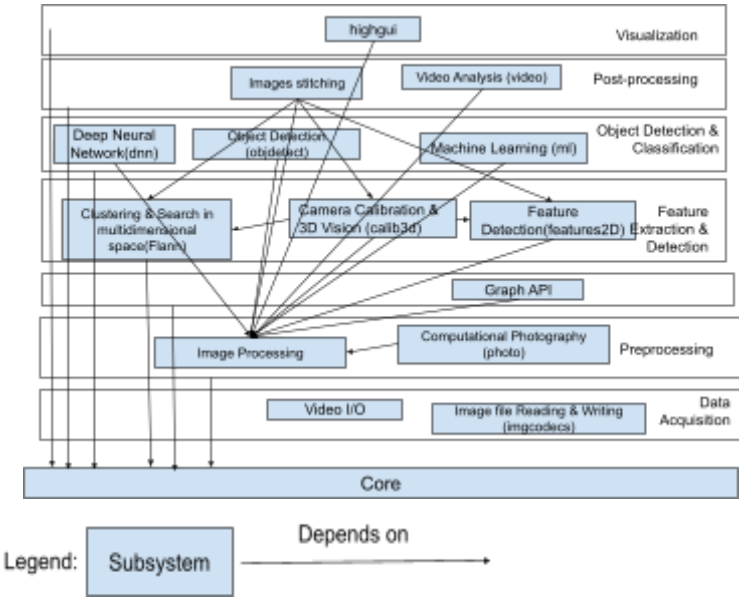

Figure 5: Conceptual Architecture of OpenCV with the modular dependencies.

Evolution of the System:

2.1 OpenCV 1.x (2006)

OpenCV 1.x was the first release of the library which was built primarily in C. The Windows installer was now shipped with a precompiled Python module providing support for Python users [28]. Key features were added such as image inpainting which filled in the missing/damaged parts of an image [28]. New image formats like JPEG2000 and OpenEXR were added through HighGUI which was responsible for image I/O as well as video capture [28]. A couple more samples like calibrate.cpp and letter_recog.cpp were introduced helping users learn new features [28]. However, one of the main issues was the limited maintainability of the C-based design, which set the stage for a major redesign in later versions.

2.2 OpenCV 2.x (2009)

2.x represented a major architectural shift introducing a C++ interface for core modules [28]. Although the C API was still usable this showed that they were looking to modernize the program. One of the biggest changes was the new cv::Mat object which made handling images easier and automatically managed memory [28]. CMake became the primary build system allowing users to integrate OpenCV in different environments [28]. It also introduced a wide range of new algorithms for object detectors and descriptors such asFAST, MSER and LDetector [28]. The machine learning module

was expanded to make the library faster and more scalable, also introducing a new tree-based algorithm [28]. Overall, 2.x improved many things 1.x lacked further enhancing the user experience.

2.3 OpenCV 3.x (2015)

3.x was a further modernization of the library, mainly focusing on improved performance and API stabilization. A new key feature added was the Transparent API (T-API), which is a transparent GPU acceleration layer using OpenCL [25]. This meant functions like cv::Canny could automatically use OpenCL if available, eliminating the need for separate GPU modules. OpenCV 3.x also introduced deeper integration with Intel's IPP and IPPICV libraries, adding more optimized functions to deliver significant speed-ups [25]. They extended on Python and Java bindings and introduced Matlab compatibility [25]. Overall, this version made the system more organized, optimized it for users, and helped prepare the library for more features that would arrive in 4.x.

OpenCV 4.x (2018)

This is the most recent release of OpenCV which has now moved onto a C++11 Library requiring a C++11-compliant compiler and at least a CMake version of 3.5.1 [26]. To further modernize the program, the majority of the legacy C functions that were introduced in 1.x were removed [26]. This simplified maintenance and reduced architectural complexity. Persistence was also reimplemented in C++ as a result of their shift away from C [26]. A new G-API was added which let users describe image processing as a graph which made it easier to optimize pipelines [26]. One of the biggest updates was to the dnn module, which integrated OpenVINO's Deep Learning Deployment Toolkit. With OpenVINO integration, models could be optimized specifically for Intel hardware resulting in performance improvements [26]. The Kinect Fusion algorithm was also implemented which builds a 3D model of a scene using the depth and color data recorded by an RGB-D camera [26]. Overall, 4.x fully transitioned everything to C++ introducing new features for users as well as optimizing previous ones.

OpenCV 5.x Alpha (2024)

5.x is the next upcoming major version of the library however it is currently in Alpha and not ready for production use yet. It now requires C++ 17 as a minimum with plans to make it also compatible with C++ 20 and C++ 23 [27]. To further clean up the program, C API has been fully removed and Python 2 support has been dropped now requiring Python 3 [27]. Another key focus is the continued expansion of the dnn module improving its ONNX compatibility and dynamic shape support [27]. Calibration and 3D modules have also been upgraded with new algorithms for more efficient and robust camera modeling [27]. OpenCV 5.x represents the next iteration in the library's evolution, improving on modernization and better support for features.

Control & Data Flow between System Components:

Focusing on understanding control and data flow is the best way to analyze OpenCV's conceptual architecture. As a pipeline-based, modular framework, OpenCV enables data(images, video, streams, feature vectors) to be directed through a sequential series of clearly defined steps, and control is coordinated largely through application-level logic.

Data Flow

Data in OpenCV is processed based on a pipeline architecture of the data flow, where each step takes in and returns data in a standard format. It is made possible by the underlying data structure, the Mat object in C++(and NumPy arrays in Python), a dense, multidimensional array of numbers [6]. The data flow between the system components of OpenCV will typically process pipeline flows with the first input acquisition. This is where input and output modules are used to read images and videos using cv::imread or cv::VideoCapture. The input is processed into Mat format, which forms the starting point of further processing [6]. The next step is preprocessing, where operations such as resizing, conversion of color spaces, denoising, and filtering are used. Each of the transformations creates a new Mat, allowing the building of more lengthy pipelines[29]. Then the extraction of features and processing of algorithms is next. This is when higher-level features such as edges, corners, or optical flow vectors are computed with modules like imgproc and features2d. The output can be a list of features/keypoints or annotated images. The next step is machine learning, which allows information to be entered into the Machine Learning(ml) module or into the Deep Neural Network (dnn) module [16]. This step converts raw features into predictions, classifications, or decisions. Lastly, output and visualization occur where the output is returned in the form of labeled frames, classification labels, or stored output. The design of this pipeline implies that the flow of data is very linear, with some branching, for example, one frame being processed by more than one algorithm at a time.

Control Flow

Although data flow is processed in sequence, control flow is handled with more flexibility, such as with event-driven control and graph execution. OpenCV modules are not independent, but rather the order of operations is specified by the user-written code, such as in C++, Python, or Java. As an example, in a face detection pipeline, the application chooses whether to capture a frame, invoke preprocessing functions, invoke a detection algorithm, and show results. With real-time applications such as video processing, control is frequently event-driven; the main loop will wait until a new frame is received, then process it, and repeat[30]. There are feedback loops in which the output frame is used to process the next frame, like updating a tracking window. OpenCV also has an execution module called Graph API(G-API), where a computation graph is defined by the user, explaining dependencies of the data[22]. Backend(CPU, GPU, OpenCL) compiles the graph, optimizes it, and then executes it. This is a change in control where user-written loops are not used, but a graph compiler that identifies the order of execution[22].

Concurrency in the System:

OpenCV uses built in native support for concurrency through parallelization. Specifically, using the cv::parallel_for_function. This enables parallelization for tasks such as image representation and processing. It uses the function setNumThreads() to configure the number of threads for the upcoming parallel operations [1]. If the threads are equal to 1, then OpenCV disables the threading and runs the functions sequentially [1]. OpenCV makes the attempt to run using the number set in setNumThreads, however, the execution is dependent on the framework [1].

TBB	“User-defined parallel constructions will run with the same threads number, if another is not specified. If later on user creates his own scheduler, OpenCV will use it”[1].
OpenMP	“No special defined behaviour”[1].
Concurrency	“If threads == 1, OpenCV will disable threading optimizations and run its functions sequentially”[25].
GCD	“Supports only values <= 0” [1].
C	“Special defined behaviour” [1].

If there is no backend support available, it will execute sequentially.

In an example of image representation, OpenCV uses parallelization methods for its execution [2]. Each pixel is stored as an element in a matrix [2]. All operations on the pixels are computed using parallelization by inheriting from a class called ParallelLoopBody [2]. Another call is then made using the range which represents the total number of pixel operations [2]. The range is then split to distribute the computations by using setNumThreads() [2].

OpenCV’s concurrency model allows efficient execution of many operations by running tasks through parallelization [2]. Through the use of cv::parallel_for_, ParallelLoopBody, and setNumThreads(), OpenCV distributes work across available threads to maximize performance [2]. However, if threading is disabled or not supported by the backend, operations will run sequentially [2]. This parallelization approach ensures that OpenCV maintains efficiency while improving the speed and responsiveness of large processing tasks.

Division of Responsibilities Among Developers:

There are 5 stakeholders who have developed OpenCV the most, have shown interest in the software, and have contributed to what OpenCV is today. The 5 stakeholders are the communicator, developer, tester, user, and integrator [31]. These are the 5 main stakeholders, while there are others, those other stakeholders aren’t applicable to OpenCV. There are in total 11 stakeholders, but we only consider the 5 that were mentioned since the web app’s design is the only thing that is considered applicable only to OpenCV [31]. This was mentioned in the book by Rozanski and Woods [31]. GitHub is where OpenCV’s source code is mainly found, and GitHub is also where all the tracking for issues, interactions, collaborations, and version control happens. OpenCV integrates its source code with GitHub. This is where stakeholders mainly come to access or use something from OpenCV or to help develop OpenCV further.

The first stakeholder is the communicator. Maksim Shabunin was the main person responsible for this role, as well as the rest of the community, who used OpenCV, also had a role in making sure OpenCV was maintained properly [31]. Maksim

Shabunin's main duties were to document OpenCV and make sure it was well documented [31]. His job was to make sure the documentation was of high quality and that everything was well organized [31]. The community would maintain the documentation of OpenCV through the use of GitHub. OpenCV's documentation, being a process initiated by the community, confirms that documentation will grow as the codebase grows along with it. As a result, OpenCV grows bigger and bigger as time goes on, as more users start using it.

Developers are the next stakeholders. There are 3 main people who have contributed the most to the development of OpenCV. They are Alexander Alekhin, Steve Puttemans, and Maksim Shabunin [31]. These 3 are the main developers, but there are many other people who have also contributed to OpenCV. There are more than 533 users who have contributed to it [31]. These users, while developing OpenCV, faced a bug when they were using the library, so they had to come up with a solution to fix that bug [31]. They would help other developers who had the same problems that they once had, and they would be able to impact what changes come in the next release of OpenCV [31]. The role of the developer is to interact with the GitHub workflows. They would write up their version of the improved code and submit it by pull request. If they noticed any bugs in the code or in the software, then they would report them, and through issues they would make additional new requests to add brand new features to the software. This entire operation creates a community-driven development that people who contribute to OpenCV almost always attempt to answer problems that they face while developing their projects.

The next stakeholder is the tester. While there is no single or group tester who was responsible for this role, developers were the ones who would actually do this [31]. The developers, when they're making a new functionality to implement for OpenCV, would create their own test that would be run by a bot called OpenCV PushBot [31]. If a pull request is accepted by an integrator, then the bot will automatically run the tests the developers wrote to ensure everything works properly [31].

The user is the next stakeholder. There are a variety of different types of users. Users include software engineers, users from large companies, users from small companies, and government agencies [31]. Some major users who have used OpenCV are IBM, Google, Intel, and many other big companies [31]. To showcase how many users actually used OpenCV, there are more than seven million downloads of OpenCV [31]. This is an estimated number, but it shows the massive number of users who use it and rely on it. The way users actually use OpenCV is by going to GitHub and downloading the source code from it, or they would go to the main site of OpenCV and SourceForge to be able to acquire the binary distributions that are already precompiled. What these binaries do is they provide users a simple way to integrate OpenCV into whatever project they are working on, for users who don't want to change the source code. This can be done very fast. These users described however, are considered passive users, but there is a large number of users who are considered active users. These users report any issues they find, document the code or enhance the documentation that already exists, and some submit their own code that they wrote themselves. This is how these active users mainly contribute to OpenCV. These users are considered to be a crucial part of the development of OpenCV and its community.

The final stakeholder is the integrator. The main 3 integrators are also the same 3 main developers of OpenCV. Those 3 people are Alexander Alekhin, Steve Puttemans, and Maksim Shabunin [31]. These 3 people attempted to develop OpenCV in a systematic way [31]. They wanted to do it in a way that keeps the code high quality, upholds the architecture, and keeps the code style consistent throughout the development process [31]. However, they faced many invalid issues as well as recurring issues [31]. These issues account to be half of OpenCV's issues, which ultimately led them to become closed issues [31]. The integrator's main role is to monitor the pull requests made on GitHub that were submitted by developers. Integrators would essentially just manage these pull requests and review them to make sure everything is good and proper. They also make sure that contributions are of high quality and meet the required criteria. They also ensure that it complies with the architectural standard set by the project. Also, contributions should not bring regressions, so this is another thing the integrator would keep an eye on. The integrator's role is essential as they are the ones who make sure the integrity of the codebase is well maintained. They also ensure OpenCV grows more and more as it's expected to, as well as making sure it doesn't stall or decline in growth.

These 5 stakeholders share the responsibility for where OpenCV is now and where it's going to be in the future. These stakeholders also use each other's strengths and abilities to create a strong, large community while making sure the code's quality is kept very high and keeping everything consistent in how everything is done. They also ensure the code is well-documented and it is thoroughly tested to ensure that OpenCV continues to evolve as time goes on.

Sequence Diagrams Based on Conceptual Architecture:

OpenDataCam : OpenDataCam implements a pipeline architecture for real-time object detection and counting. Video data flows sequentially through the camera, detection, tracking, and visualization stages [32].

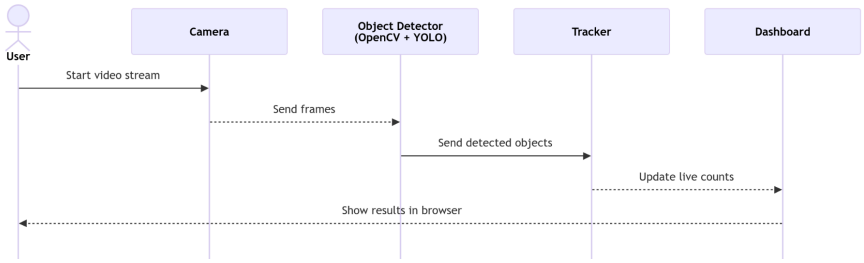

Figure 6. Sequence Diagram, adapted from https://github.com/opendatacam/opendatacam [32].

Computer Vision in Robotics

Description: This example illustrates how OpenCV is integrated within a ROS2-based vision system to guide robot movement. The pipeline consists of image capture, processing, control decision, and actuation [33].

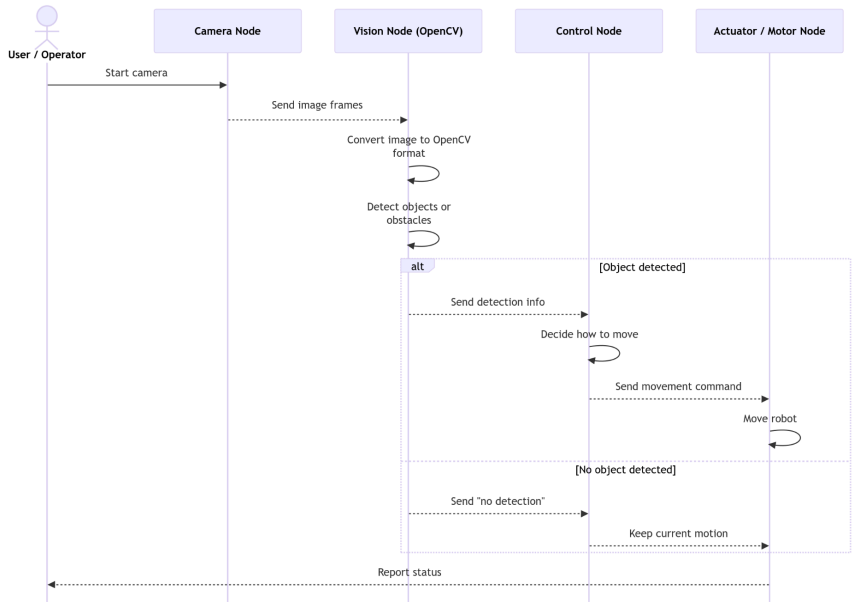

Figure 7. Sequence Diagram of OpenCV use case in robotics, adapted from https://ibrahimmansur4.medium.com/integrating-opencv-with-ros2-a-comprehensive-guide-to-computer-vision-in-robotics-66b97fa2de92[33]

Conclusions:

In this report, we explored the conceptual architecture of OpenCV and the modular,layered and extensible nature of the architecture that has helped it become one of the most used computer vision libraries in the industry. All the modules, including the essential matrix and utility structures up to sophisticated subsystems of deep learning, processing visual images, and 3D vision, are part of a scalable architecture that can be used to solve a wide range of tasks. The history of OpenCV from a C-based library to a current C++ 17 system illustrates a definite roadmap to better performance, maintainability, and compatibility. The data and control flow analysis of OpenCV gives some emphasis on the fact that it is a pipeline-based data process, which is characterized by the smooth flow of data through the acquisition, processing, analysis, and output phases. Its concurrency model and graph-based hardware backends are based on this design. Furthermore, the breakdown of duties between the developers and integrators and the greater open-source community will help see how the OpenCV environment of collaboration has played a vital part in the ongoing development of OpenCV. To wrap everything up, the conceptual architecture of OpenCV as a layered architecture is an example of a well-developed and strong software architecture with a reasonable amount of flexibility, performance, and usability combined. Future perspectives of OpenCV can involve interconnection with AI systems, more optimization with GPUs, and further development of the ability to support real-time edge computing systems.

Lessons Learned:

A key lesson identified was that enabling parallelism in OpenCV does not always lead to performance improvements [34]. Without tuning according to machine, number of cores, etc., it may introduce overhead [34]. In future work, evaluating concurrency across multiple platforms early in the process would provide a more accurate

understanding of its real-world behavior [27]. The conceptual diagram was adjusted following the presentation due to the feedback received.The new conceptual diagram was derived from the steps in the Computer Vision Pipeline. The key lesson was that in order to construct the conceptual architectural diagram, it is important to focus on the goal of the system and the existing documentation.However, relying on function-level documentation or code should be avoided, as the diagram should remain high-level and focused on overall system structure rather than implementation details.

References:

[1] "OpenCV: Utility and system functions and macros," Opencv.org, 2025. https://docs.opencv.org/4.x/db/de0/group__core__utils.html#gae78625c3c2aa9e0b83ed31b73c6549c0 (accessed Oct. 07, 2025).

[2] "OpenCV: How to use the OpenCV parallel_for_ to parallelize your code," Opencv.org, 2025. https://docs.opencv.org/4.x/d7/dff/tutorial_how_to_use_OpenCV_parallel_for_.html

[3] "About," OpenCV. Accessed: Oct. 01, 2025. [Online]. Available: https://opencv.org/about/

[4] F. Alvi, "Why You Need To Start Learning OpenCV in 2025!," OpenCV. Accessed: Oct. 10, 2025. [Online]. Available: https://opencv.org/blog/learning-opencv/

[5] "What are the main steps in a typical Computer Vision Pipeline?," GeeksforGeeks. Accessed: Oct. 10, 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-vision/what-are-the-main-steps-in-a-typical-computer-vision-pipeline/

[6] "OpenCV: Introduction." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d1/dfb/intro.html

[7] "What is OpenCV? A Guide for Beginners." Accessed: Oct. 06, 2025. [Online]. Available: https://blog.roboflow.com/what-is-opencv/

[8] "OpenCV: Core functionality." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d0/de1/group__core.html

[9] "OpenCV: cv::Mat Class Reference." Accessed: Oct. 10, 2025. [Online]. Available: https://docs.opencv.org/4.x/d3/d63/classcv_1_1Mat.html

[10] "OpenCV: Image Processing." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d7/dbd/group__imgproc.html

[11] "OpenCV: Video I/O." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/dd/de7/group__videoio.html

[12] "OpenCV: Video I/O with OpenCV Overview." Accessed: Oct. 10, 2025. [Online]. Available: https://docs.opencv.org/3.4/d0/da7/videoio_overview.html

[13] "OpenCV: Camera Calibration and 3D Reconstruction." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html

[14] "Feature extraction and image classification using OpenCV - GeeksforGeeks." Accessed: Oct. 10, 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-vision/feature-extraction-and-image-classification-using-opencv/

[15] "OpenCV: Clustering and Search in Multi-Dimensional Spaces." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/dc/de5/group__flann.html

[16] "OpenCV: Deep Neural Network module." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d6/d0f/group__dnn.html

[17] “How can you preprocess image data for a neural network?” Accessed: Oct. 10, 2025. [Online]. Available: https://www.linkedin.com/advice/0/how-can-you-preprocess-image-data-neural-network-a4n9c
[18] “LinkedIn.” Accessed: Oct. 10, 2025. [Online]. Available: https://www.linkedin.com/pulse/understanding-image-types-transformation-machine-learning-9zsnc
[19] “OpenCV: Computational Photography.” Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d1/d0d/group__photo.html
[20] “OpenCV: Images stitching.” Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d1/d46/group__stitching.html
[21] “Introduction to Object Detection Using Image Processing - GeeksforGeeks.” Accessed: Oct. 10, 2025. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/introduction-to-object-detection-using-image-processing/
[22] “OpenCV: High-level GUI.” Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d7/dfc/group__highgui.html
[23] “OpenCV: High-level design overview.” Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/de/d4d/gapi_hld.html
[24] “OpenCV: Graph API.” Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d0/d1e/gapi.html
[25] “OpenCV 3.0.” Accessed: Oct. 07, 2025. [Online]. Available: https://opencv.org/blog/opencv-3-0/
[26] “OpenCV 4.0.” Accessed: Oct. 07, 2025. [Online]. Available: https://opencv.org/blog/opencv-4-0/
[27] “OpenCV Change Logs: Version 4.12.0.” Accessed: Oct. 07, 2025. [Online]. Available: https://github.com/opencv/opencv/wiki/OpenCV-Change-Logs#version4120
[28] “OpenCV Change Logs: v1.0–v2.1.” Accessed: Oct. 07, 2025. [Online]. Available: https://github.com/opencv/opencv/wiki/OpenCV-Change-Logs-v1.0%E2%80%90v2.1
[29] “Computer Vision Pipeline Architecture: A Tutorial | Toptal®,” *Toptal Engineering Blog*. https://www.toptal.com/computer-vision/computer-vision-pipeline
[30] Dmitry Matveev, “Hybrid CV/DL pipelines with OpenCV 4.4 G-API,” *OpenCV*, Jul. 22, 2020. https://opencv.org/blog/hybrid-cv-dl-pipelines-with-opencv-4-4-g-api/ (accessed Oct. 07, 2025).
[31] “OpenCV: Stakeholders.” Accessed: Oct. 07, 2025. [Online]. Available: https://delftswa.gitbooks.io/desosa2016/content/opencv/Chapter.html
[32] “opendatacam / opendatacam,” Accessed Oct. 07, 2025. [Online]. Available: https://github.com/opendatacam/opendatacam
[33] I. B. Mansur, “Integrating OpenCV with ROS2” Accessed Oct. 07, 2025. [Online]. Available: https://ibrahimmansur4.medium.com/integrating-opencv-with-ros2-a-comprehensive-guide-to-computer-vision-in-robotics-66b97fa2de92
[34] Rachit Tayal, “A Note on OpenCV Threads & Performance in Prod.,” Medium, May 14, 2019. https://medium.com/@rachittayal7/a-note-on-opencv-threads-performance-in-prod-d10180716fba (accessed Oct. 09, 2025).